

LAB3

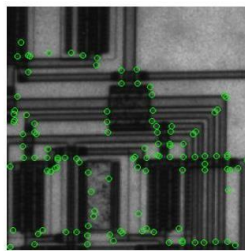
Features descriptors, local feature matching

1. Local Feature Detection and Extraction

Local features and their descriptors, which are a compact vector representation of a local neighborhood, are the building blocks of many computer vision algorithms. Their applications include image registration, object detection and classification, tracking, and motion estimation. Using local features enables these algorithms to better handle scale changes, rotation, and occlusion.

What Are Local Features?

Local features refer to a pattern or distinct structure found in an image, such as a point, edge, or small image patch. They are usually associated with an image patch that differs from its immediate surroundings by texture, color, or intensity. Local features include blobs, corners, and edge pixels.



Benefits and Applications of Local Features:

Local features facilitate finding image correspondences regardless of occlusion, changes in viewing conditions, or the presence of clutter. Besides, the properties of local features make them suitable for image classification, such as in “Image Classification with Bag of Visual Words”. Local features are used in two fundamental ways:

- Image Matching and Correspondence.

Example:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load two example images (replace with your own image paths)
image1 = cv2.imread('i1.jpg', cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread('i2.jpg', cv2.IMREAD_GRAYSCALE)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors for both images
```

```

keypoints1, descriptors1 = sift.detectAndCompute(image1, None)
keypoints2, descriptors2 = sift.detectAndCompute(image2, None)

# Create a Brute-Force Matcher
bf = cv2.BFMatcher()

# Match descriptors between the two images
matches = bf.knnMatch(descriptors1, descriptors2, k=2)

# Apply ratio test to obtain good matches
good_matches = []
for m, n in matches:
    if m.distance < 0.5 * n.distance:
        good_matches.append(m)

# Draw matches on a new image
matched_image = cv2.drawMatches(image1, keypoints1, image2, keypoints2, good_matches, None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

# Display the matched image
plt.imshow(matched_image), plt.title('Image Matching Result')
plt.show()

```



- Object Detection and Recognition.

Example:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("input.jpg")
original_image = image.copy() # Make a copy for displaying the original image

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

```

```

# Apply GaussianBlur to reduce noise and improve contour detection
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Use HoughCircles to detect circles
circles = cv2.HoughCircles(
    blurred, cv2.HOUGH_GRADIENT, dp=1, minDist=20, param1=50, param2=30, minRadius=10,
    maxRadius=50
)

if circles is not None:
    circles = np.uint16(np.around(circles))
    for i in circles[0, :]:
        # Draw the outer circle
        cv2.circle(image, (i[0], i[1]), i[2], (0, 255, 0), 2)
        # Draw the center of the circle
        cv2.circle(image, (i[0], i[1]), 2, (0, 0, 255), 3)

# Display the original and LBP images
plt.figure(figsize=(10, 7))

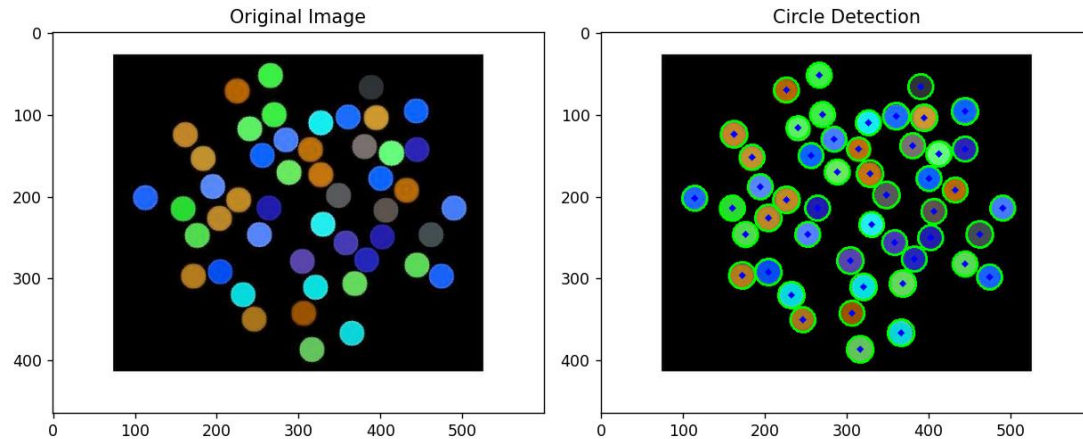
plt.subplot(1, 2, 1)
plt.imshow(original_image, cmap='gray')
plt.title("Original Image")

plt.subplot(1, 2, 2)
plt.imshow(image, cmap='gray')
plt.title("Circle Detection")

plt.tight_layout()
plt.show()

cv2.waitKey(0)
cv2.destroyAllWindows()

```



What Makes a Good Local Feature?

Detectors that rely on gradient-based and intensity variation approaches detect good local features. These features include edges, blobs, and regions. Good local features exhibit the following properties:

- **Repeatable detections:** When given two images of the same scene, most features that the detector finds in both images are the same. The features are robust to changes in viewing conditions and noise.
- **Distinctive:** The neighborhood around the feature center varies enough to allow for a reliable comparison between the features.
- **Localizable:** The feature has a unique location assigned to it. Changes in viewing conditions do not affect its location.

Feature Detection vs. Feature Extraction

- Feature detection selects regions of an image that have unique content, such as corners or blobs. Use feature detection to find points of interest that you can use for further processing. These points do not necessarily correspond to physical structures, such as the corners of a table.

Example:

```
import cv2
import numpy as np

# Load an image from file
image = cv2.imread('01.png', cv2.IMREAD_GRAYSCALE)

# Initialize a feature detector (e.g., ORB - Oriented FAST and Rotated BRIEF)
feature_detector = cv2.ORB_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = feature_detector.detectAndCompute(image, None)

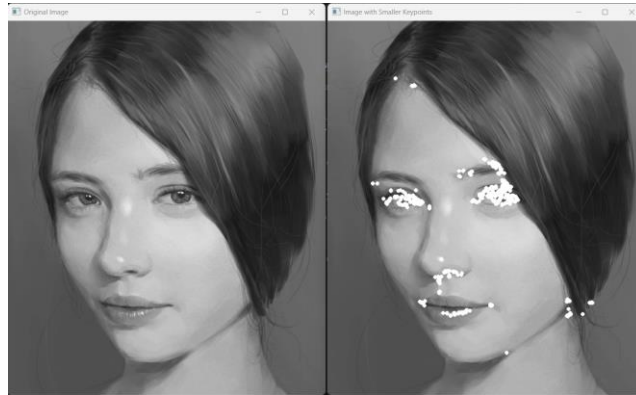
# Draw smaller keypoints on the image
image_with_keypoints = np.copy(image) # Create a copy of the image
```

```

for kp in keypoints:
    x, y = map(int, kp.pt)
    cv2.circle(image_with_keypoints, (x, y), 2, (255, 125, 125), 2)

# Display the original and the image with smaller keypoints
cv2.imshow('Original Image', image)
cv2.imshow('Image with Smaller Keypoints', image_with_keypoints)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



- Feature extraction involves computing a descriptor, which is typically done on regions centered on detected features. Descriptors rely on image processing to transform a local pixel neighborhood into a compact vector representation. This new representation permits comparison between neighborhoods regardless of changes in scale or orientation.

Example:

```

import cv2

# Load an image from file
image = cv2.imread('01.png', cv2.IMREAD_GRAYSCALE)

# Initialize the SIFT detector
sift = cv2.SIFT_create()

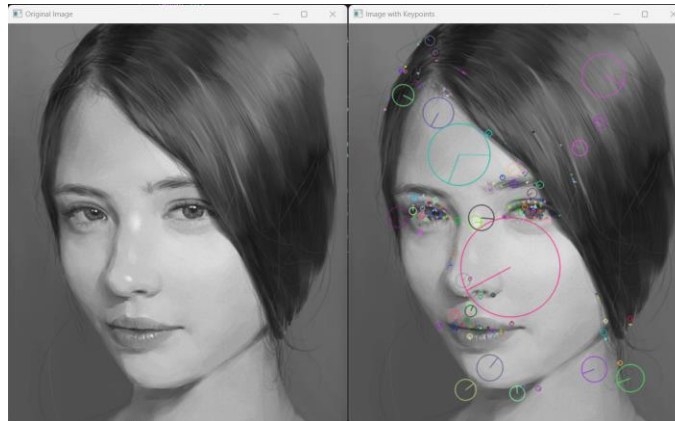
# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(image, None)

# Print the number of keypoints and the size of each descriptor
print("Number of Keypoints:", len(keypoints))
print("Size of Each Descriptor:", descriptors.shape)

# Display the original image with keypoints
image_with_keypoints = cv2.drawKeypoints(image, keypoints, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
# Display the original and the image with smaller keypoints

```

```
cv2.imshow('Original Image', image)
# Display the original and the image with Keypoints
cv2.imshow('Image with Keypoints', image_with_keypoints)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Considerations for Selecting a Detector and Descriptor

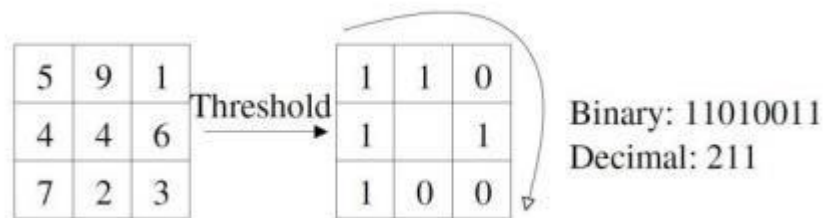
Select the best feature detector and descriptor by considering the criteria of your application and the nature of your data.

Criteria	Suggestion
Type of features in your image	Use a detector appropriate for your data. For example, if your image contains an image of bacteria cells, use the blob detector rather than the corner detector. If your image is an aerial view of a city, you can use the corner detector to find man-made structures.
Context in which you are using the features: - Matching key points - Classification	The HOG, SURF, and KAZE descriptors are suitable for classification tasks. In contrast, binary descriptors, such as ORB, BRISK and FREAK, are typically used for finding point correspondences between images, which are used for registration.
Type of distortion present in your image	Choose a detector and descriptor that addresses the distortion in your data. For example, if there is no scale change present, consider a corner detector that does not handle scale. If your data contains a higher level of distortion, such as scale and rotation, then use SURF, ORB or KAZE feature detector and descriptor. The SURF and the KAZE methods are computationally intensive.
Performance requirements: - Real-time performance required - Accuracy versus speed	Binary descriptors are generally faster but less accurate than gradient-based descriptors. For greater accuracy, use several detectors and descriptors at the same time.

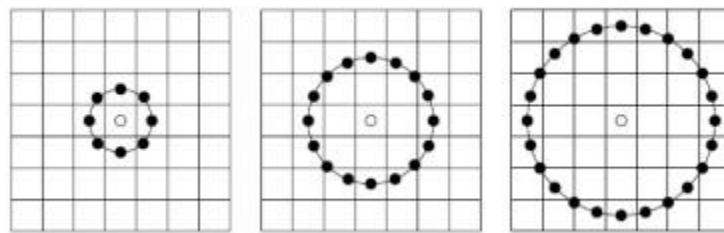
2. Local binary pattern (LBP)

- It is a simple but highly effective descriptive technique for classifying objects within computer vision that filters out adjacent pixels using determined considerations and obtains a representative binary value. Encodes the ratio of the central pixel to the color intensity of neighboring pixels.

- Due to its high discriminatory capacity, it constitutes a common approach for solving a multitude of problems. Probably one of its most important characteristics is the robustness of its invariant to light variations.
- The original LBP operator labels the pixels of an image with decimal numbers, called Local Binary Patterns or LBP codes, which encode the local structure around each pixel. It proceeds thus, as illustrated in the figure shown below.
- Each pixel is compared with its eight neighbors in a 3x3 neighborhood by subtracting the center pixel value; The resulting strictly negative values are encoded with 0 and the others with 1; A binary number is obtained by concatenating all these binary codes in a clockwise direction starting from the top-left one and its corresponding decimal value is used for labeling. The derived binary numbers are referred to as Local Binary Patterns or LBP codes.



- One limitation of the basic LBP operator is that its small 3x3 neighborhood cannot capture dominant features with large scale structures. To deal with the texture at different scales, the operator was later generalized to use neighborhoods of different sizes. A local neighborhood is defined as a set of sampling points evenly spaced on a circle which is centered at the pixel to be labeled, and the sampling points that do not fall within the pixels are interpolated using bilinear interpolation, thus allowing for any radius and any number of sampling points in the neighborhood.
- The following figure shows some examples of the extended LBP operator, where the notation (P, R) denotes a neighborhood of P sampling points on a circle of radius of R.



Examples of the extended LBP operator: the circular (8, 1), (16, 2), and (24, 3) neighborhoods.

Example:

```
import numpy as np
from skimage import feature, io, exposure
import matplotlib.pyplot as plt
# A function that takes an image, a radius, and the number of neighbors as parameters and returns the Local Binary Pattern (LBP) of the image.
def extended_lbp(image, radius, neighbors):
    lbp = feature.local_binary_pattern(image, neighbors, radius, method='uniform')
```



```

    return lbp
# Load an example grayscale image
image_path = '2.jpg'
original_image = io.imread(image_path, as_gray=True)
# Convert the image to a numpy array if it's not already
if not isinstance(original_image, np.ndarray):
    original_image = np.array(original_image)
# Ensure image is in the range [0, 1]
normalized_image = exposure.rescale_intensity(original_image, in_range=(0, 1))
# Convert the image to uint8 for LBP computation
image_uint8 = (normalized_image * 255).astype(np.uint8)
# Circular LBP with (8, 1) neighborhood
lbp_8_1 = extended_lbp(image_uint8, radius=1, neighbors=8)
# Circular LBP with (16, 2) neighborhood
lbp_16_2 = extended_lbp(image_uint8, radius=2, neighbors=16)
# Circular LBP with (24, 3) neighborhood
lbp_24_3 = extended_lbp(image_uint8, radius=3, neighbors=24)
# Display the original and LBP images
plt.figure(figsize=(10, 7))

plt.subplot(2, 2, 1)
plt.imshow(normalized_image, cmap='gray')
plt.title('Original Image')

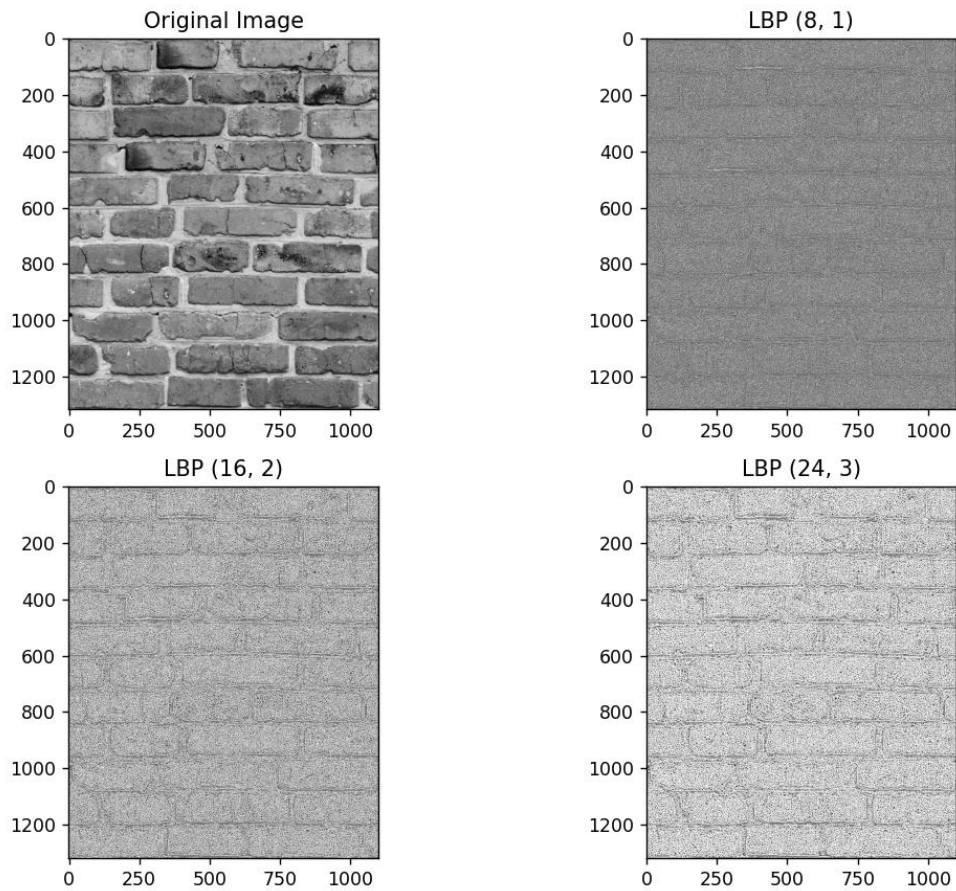
plt.subplot(2, 2, 2)
plt.imshow(lbp_8_1, cmap='gray')
plt.title('LBP (8, 1)')

plt.subplot(2, 2, 3)
plt.imshow(lbp_16_2, cmap='gray')
plt.title('LBP (16, 2)')

plt.subplot(2, 2, 4)
plt.imshow(lbp_24_3, cmap='gray')
plt.title('LBP (24, 3)')

plt.tight_layout()
plt.show()

```

Extract LBP features from the images to encode their texture information:

```
import cv2
from skimage import feature
import numpy as np
import os

# Function to compute LBP histogram for an image
def compute_lbp_histogram(image):
    # Compute LBP features using scikit-image
    lbp = feature.local_binary_pattern(image, P=8, R=1, method="uniform")

    # Calculate histogram of LBP features
    hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, 10), range=(0, 9))

    # Normalize the histogram to obtain the feature vector
    hist = hist.astype("float")
    hist /= (hist.sum() + 1e-7)

    return hist

# Function to extract LBP features from a directory of images
```

```

def extract_lbp_features(directory):
    features = []
    labels = []

    for filename in os.listdir(directory):
        if filename.endswith(".jpg") or filename.endswith(".png"):
            image_path = os.path.join(directory, filename)

            # Load image using OpenCV
            image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

            # Resize image if needed
            # image = cv2.resize(image, (100, 100))

            # Compute LBP histogram for the image
            hist = compute_lbp_histogram(image)

            # Append the feature vector and label
            features.append(hist)
            labels.append(filename.split('_')[0]) # Assuming the label is the prefix before '_'

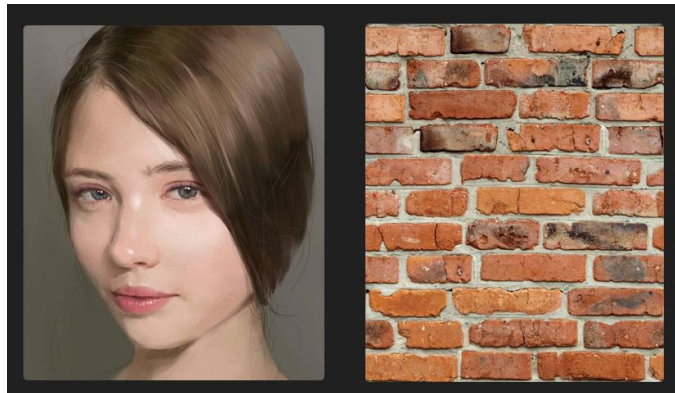
    return np.array(features), np.array(labels)

# Example usage
image_directory = 'images'
features, labels = extract_lbp_features(image_directory)

# Print the extracted features and labels
print("LBP Features:")
print(features)
print("Labels:")
print(labels)

```

Images:



Output:

```
LBP Features:
[[0.04434208 0.06895458 0.03089367 0.07773412 0.13760297 0.11688435
 0.05769835 0.0820995 0.38379038]
 [0.06970065 0.09066147 0.07379018 0.10964202 0.14353704 0.11660022
 0.08502483 0.08984205 0.22120155]]
Labels:
['1.jpg' '2.jpg']
```

Gauge the similarity between the LBP features by computing the squared error :

Example:

```
import cv2
from skimage import feature
import numpy as np

# Function to compute LBP histogram for an image
def compute_lbp_histogram(image):
    # Compute LBP features using scikit-image
    lbp = feature.local_binary_pattern(image, P=8, R=1, method="uniform")

    # Calculate histogram of LBP features
    hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, 10), range=(0, 9))

    # Normalize the histogram to obtain the feature vector
    hist = hist.astype("float")
    hist /= (hist.sum() + 1e-7)

    return hist

# Function to compute squared error between two LBP feature vectors
def compute_squared_error(feature1, feature2):
    return np.sum((feature1 - feature2) ** 2)

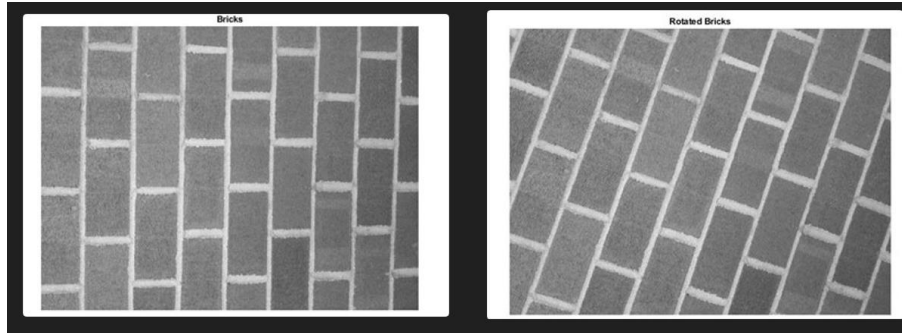
# Load two images for comparison
image1 = cv2.imread('Picture1.jpg', cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread('Picture2.jpg', cv2.IMREAD_GRAYSCALE)

# Compute LBP features for the two images
image1_feature = compute_lbp_histogram(image1)
image2_feature = compute_lbp_histogram(image2)

# Compute squared error between the LBP features
error = compute_squared_error(image1_feature, image2_feature)

# Print the result
print(f"Squared Error between Image 1 and Image 2: {error}")
```

Images:



Output:

```
PS E:\course 2023-2024\LAB3> python '.\Gauge the similarity.py'
Squared Error between Image 1 and Image 2: 0.0011249426550911806
PS E:\course 2023-2024\LAB3> █
```

3. Histogram of oriented gradients (HOG) features

- In the HOG feature descriptor, the distribution (histograms) of directions of gradients (oriented gradients) are used as features.

Example:

```
import cv2
from matplotlib import pyplot as plt

# Read the image
image = cv2.imread('Picture1.jpg', cv2.IMREAD_GRAYSCALE)

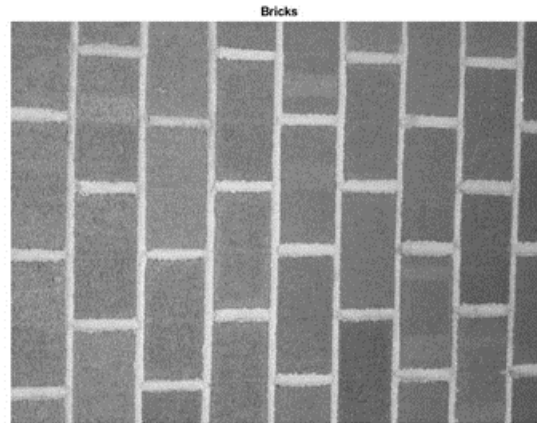
# Initialize HOG descriptor
hog = cv2.HOGDescriptor()

# Compute HOG features
features = hog.compute(image)

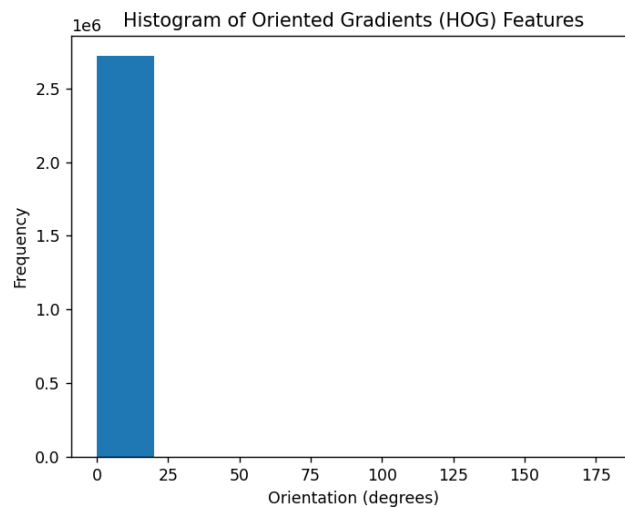
# Reshape features for visualization
features_reshaped = features.reshape(-1, 1)

# Plot the HOG features as a histogram
plt.hist(features_reshaped, bins=9, range=(0, 180))
plt.title('Histogram of Oriented Gradients (HOG) Features')
plt.xlabel('Orientation (degrees)')
plt.ylabel('Frequency')
plt.show()
```

Images:



Output:



- Gradients (x and y derivatives) of an image are useful because the magnitude of gradients is large around edges and corners (regions of abrupt intensity changes) and we know that edges and corners pack in a lot more information about object shape than flat regions.

Calculating the Gradient Images:

To calculate a HOG descriptor, we need to first calculate the horizontal and vertical gradients; after all, we want to calculate the histogram of gradients. This is easily achieved by filtering the image with the following kernels.

-1	0	1
----	---	---

-1
0
1

We can find the magnitude and direction of the gradient using the following formula:

$$g = \sqrt{g_x^2 + g_y^2}$$

$$\theta = \arctan \frac{g_y}{g_x}$$

Example:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read the image
image = cv2.imread('i1.jpg', cv2.IMREAD_GRAYSCALE)

# Compute gradients in x and y directions
grad_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=5)
grad_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=5)

# Calculate the magnitude and direction of gradients
magnitude, angle = cv2.cartToPolar(grad_x, grad_y, angleInDegrees=True)

# Create a 2x2 subplot grid
fig, axes = plt.subplots(2, 2, figsize=(10, 10))

# Original image
axes[0, 0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axes[0, 0].set_title('Original Image')

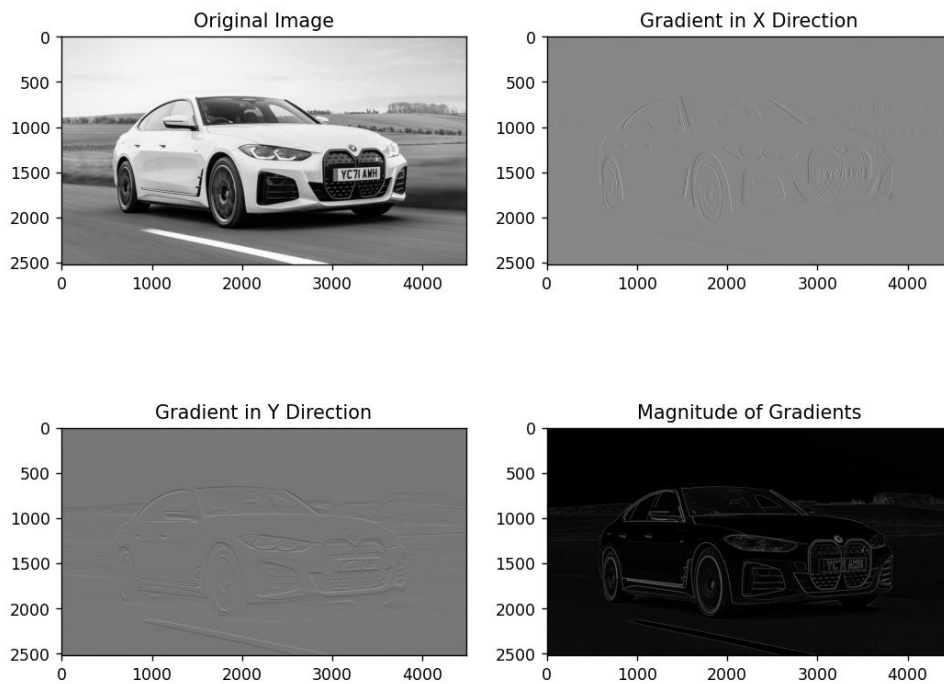
# Gradients in x and y directions
axes[0, 1].imshow(grad_x, cmap='gray')
axes[0, 1].set_title('Gradient in X Direction')

axes[1, 0].imshow(grad_y, cmap='gray')
axes[1, 0].set_title('Gradient in Y Direction')

# Magnitude of gradients
axes[1, 1].imshow(magnitude, cmap='gray')
axes[1, 1].set_title('Magnitude of Gradients')

plt.show()
```

Image and output:



Calculating the HOG feature vector

Example:

```
import cv2
from skimage.feature import hog
from skimage import exposure
import numpy as np

# Read the image
image = cv2.imread('i1.jpg', cv2.IMREAD_GRAYSCALE)
# Calculate HOG features
features, hog_image = hog(image, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2), block_norm='L2-Hys', visualize=True)
# Reshape HOG features for block normalization
features_resaped = features.reshape((image.shape[0] // 8 - 1, image.shape[1] //
8 - 1, -1))

# Apply block normalization
normalized_features = []
for i in range(features_resaped.shape[0] - 1):
    for j in range(features_resaped.shape[1] - 1):
        block = features_resaped[i:i+2, j:j+2, :].flatten()
        block /= np.linalg.norm(block, 2)
```



```

        normalized_features.extend(block)

normalized_features = np.array(normalized_features)

# Reshape normalized features for final HOG feature vector
final_feature_vector = normalized_features.flatten()

print("Final HOG Feature Vector Shape:", final_feature_vector.shape)

```

Image:



Output:

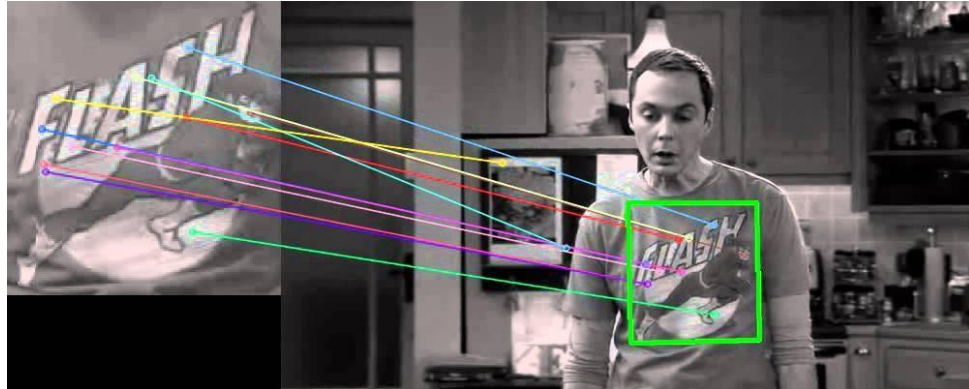
```

Final HOG Feature Vector Shape: (25150176,)

```

4. SURF (Speeded-Up Robust Features)

- The SURF method (Speeded Up Robust Features) is a fast and robust algorithm for local, similarity invariant representation and comparison of images.
- The main interest of the SURF approach lies in its fast computation of operators using box filters, thus enabling real-time applications such as tracking and object recognition.



SURF is composed of two steps:

- Feature Extraction: The approach for interest point detection uses a very basic Hessian matrix approximation.
- Feature Description: The creation of SURF descriptor takes place in two steps. The first step consists of fixing a reproducible orientation based on information from a circular region around the keypoint. Then, we construct a square region aligned to the selected orientation and extract the SURF descriptor from it.

5. exercises:

Task 1: Feature Extraction with SIFT

Objective:

Extract and visualize key features from two images using the Scale-Invariant Feature Transform (SIFT).

Steps:

- Load two grayscale images of your choice.
- Apply the SIFT feature detector to identify key points and compute descriptors for both images.
- Display the detected key points on the original images.
- Visualize the descriptors to understand the information captured by SIFT.

Task 2: Local Feature Matching with SIFT

Objective:

Perform local feature matching between two images using SIFT descriptors.

Steps:

- Load two grayscale images (different from those used in Task 1).
- Use the SIFT feature detector to find key points and compute descriptors for both images.
- Match the descriptors using a feature matching algorithm (e.g., Brute-Force Matcher).
- Apply a ratio test or another filtering method to retain only good-quality matches.
- Visualize the correspondence by drawing the matched key points between the two images.

Task 3: Local Binary Pattern (LBP) Feature Extraction

Objective:

Extract Local Binary Pattern (LBP) features from multiple texture images and prepare them for analysis.

Steps:

- Create a directory containing grayscale images with different textures.
- Implement a function that computes the LBP histogram for a given image.
- Apply the LBP extraction function to each image in the directory.
- Normalize the resulting LBP histograms to form consistent feature vectors.
- Save the extracted LBP features along with the corresponding labels for each image.

Task 4: Image Alignment Using Feature Matching

Objective:

Use SIFT key points and matches to align one image to another through geometric transformation.

Steps:

- Select two related images (e.g., the same scene captured from different viewpoints).
- Detect SIFT key points and descriptors for both images.
- Perform feature matching using Brute-Force or FLANN-based matching.
- Estimate a homography or affine transformation matrix using RANSAC to handle outliers.
- Warp one image to align with the other and display both the before-and-after alignment results.

Task 5: Texture Classification Using LBP Features

Objective:

Build a simple classifier to categorize images based on their texture features extracted using LBP.

Steps:

- Use the LBP feature vectors obtained from Task 3.
- Split the dataset into training and testing sets.
- Train a classifier (e.g., k-Nearest Neighbors, SVM, or Decision Tree) on the training set.
- Evaluate the model on the test set and report performance metrics such as accuracy and confusion matrix.
- Visualize a few correctly and incorrectly classified examples.